

System Design:

[System Design Tutorial - GeeksforGeeks](#)

**MAKE SURE TO ANALYZE TIME AND SPACE
COMPLEXITY OF EACH SOLUTION**

DSA:

Arrays

[Check if Array Is Sorted and Rotated - LeetCode](#) **Easy(Max 10 Minutes)**

1.Simple Approach Two passes

Follow Up:

2.Can you do it in a single pass?

[Rotate Array - LeetCode](#) **Easy(Max 10 Minutes)**

1.Simple Naive Approach

Follow Up:

2.Can You do it using properties of modulo?

3. Do it In place

[Missing Number - LeetCode](#) **Easy(Max 5 Minutes)**

1.Simple Approach

Follow Up:

2.Can you do it using xor?

[Move Zeroes - LeetCode](#) **Easy(Max 5-10 Minutes)**

1.Brute Force (Nested Loop)

Follow Up:

2.Can you do it in $O(N)$ Time and $O(1)$ Space?

[Maximum Subarray - LeetCode](#) **Medium(Max 20 Minutes)**

1.Try to do it with Divide and Conquer

Follow Up:

2.Can you do better? (Kadane's Algo Not intuitive but...)

[Two Sum - LeetCode](#) (Tricky You have to return indices) **Easy(Max 10 Minutes)**

1.Nested Loop

2.Using Data Structures

[Sort Colors - LeetCode](#) **Easy(Max 10 Minutes)**

- 1.Sort the Whole Array
- 2.Dutch National Flag

[Best Time to Buy and Sell Stock - LeetCode](#) **Easy(Max 10 Minutes)**

- 1.Nested Loop
- 2.Single Pass
- 3.DP (will discuss it later while doing DP qs)

[Rearrange Array Elements by Sign - LeetCode](#) **Easy(Max 15 Minutes)**

- 1.Nested Loop
- 2.O(N) space in linear time

Follow Up:

- 3.What if order doesn't matter? Can you do it in-place?

[Array Leaders | Practice | GeeksforGeeks](#) **Easy(Max 5-10 Minutes)**

- 1.Brute Force
- 2.O(N) space and O(N) Time

Follow Up:

3. Can You Get Rid of Additional Space?

[Merge Sorted Array - LeetCode](#) **Easy(Max 5-10 Minutes)**

- 1.Using Extra Space (vector)

Follow Up:

- 2.Can you do it in-place?

[Find Missing and Repeated Values - LeetCode](#) **Easy(Max 10-15 Minutes)**

- 1.Brute Force Nested Loops
- 2.HaspMaps

Follow Up:

3. Can You Come up with an O(1) Space solution?

[Max Consecutive Ones - LeetCode](#) **Easy(Max 5-10 Minutes)**

- 1.Nested Loop

Follow Up:

- 2.Can you do it in a single loop?

[Max Consecutive Ones III - LeetCode](#)*** Medium(Max 20 Minutes)

1.Brute Force (Nested Loop)

Follow Up:

2. Can you do better?

[Maximum Sum Circular Subarray - LeetCode](#) Medium(Max 20 Minutes)

1.Rotate and find all max subarray sum

2. Can you do better?

[Search in Rotated Sorted Array - LeetCode](#)*** Medium(Max 15 Minutes)

1.Linear Search

Follow Up:

2.Use the fact that somehow array is sorted

[Majority Element - LeetCode](#) Easy(Max 5-10 Minutes)

1.Brute Force ($O(N)$ & $O(N)$)

Follow Up:

2.Can we do better?

[Next Permutation - LeetCode](#) Medium(Max 15 Minutes)

1.Brute Force Generate all permutations. [Permutations - LeetCode](#)

2.Can You optimize it?

[Longest Consecutive Sequence - LeetCode](#) Medium(Max 15 Minutes)

1.Sorting $n(\log n)$

Follow Up:

2.Can you do better?

[Majority Element II - LeetCode](#) Medium(Max 15 Minutes)

1.Brute Force (Maps)

Follow Up:

2. Can you come up with a linear solution and constant space?

[3Sum - LeetCode](#) Medium(Max 15 Minutes)

1.Brute Force

2.HashMaps

Follow Up:

3.Can you come up with a better solution?

[4Sum - LeetCode](#) **Medium(Max 15-20 Minutes)**

1.Brute Force

2.HashMaps

Follow Up:

3. Can You come up with a better Solution?

[Count Inversions | Practice | GeeksforGeeks](#)

1.Brute Force

2.NlogN

[Reverse Pairs - LeetCode](#) *****

1.Brute Force

2.NlogN

[Maximum Product Subarray - LeetCode](#)

1.Brute Force (Nested Loop)

2.Double Pass

Follow Up:

3.Can You do it in a single pass without using extra space?

Must Read:

[Maximum Product Subarray - LeetCode](#)

[Trapping Rain Water - LeetCode](#)

1.Nested Loops

2.O(N) Space and Linear Time

Follow Up:

3. Can you come up with O(1) space and linear time complexity?

[Product of Array Except Self - LeetCode](#)

1.Brute Force

2.Simple Approach Total Product

3.Suffix and prefix Approach

Follow Up:

4. Can You Get Rid of Space?

Must Read:

[\[3 Minute Read\] Mimicking an Interview - Product of Array Except Self - LeetCode](#)

[Find Minimum in Rotated Sorted Array - LeetCode](#)

1. Linear Search
2. Sort and Binary Search

Follow Up:

3. Can You Come Up with $\log N$ Solution?

[Container With Most Water - LeetCode](#)

1. Brute Force

Follow Up:

2. Can You Come Up with Linear Solution and Constant Space?

[Common in 3 Sorted Arrays | Practice | GeeksforGeeks](#)

1. Brute Force (All Combinations)
2. Three Pointers (Using Data Structure for handling duplicates)

Follow Up:

3. Can you handle the duplicates without using any additional Data Structure?

[Merge Intervals - LeetCode](#)

1. Brute Force
2. Optimal Solution

[Find Pair Given Difference | Practice | GeeksforGeeks](#) (Tricky qs)

1. Brute Force
2. Extra Space (tricky)
3. Two Pointers

[Subarray Sum Equals K - LeetCode](#) (Same for Subarray with 0 sum)

1. Brute Force

Follow Up:

2. Can You reduce the Time Complexity to $O(N)$

Strings:

[String Compression - LeetCode](#) (Greedy)

1. Brute Force with Extra Space

Follow Up:

2. Can You reduce the Space Complexity to $O(1)$

[Check if the Sentence Is Pangram - LeetCode](#)

1. Brute Force with Extra Space

Follow Up:

2. Better Solution with Constant Space

[Count and Say - LeetCode](#)

1. Recursion

<https://leetcode.com/problems/integer-to-roman/description/>

1. Brute Force

[Check If Two String Arrays are Equivalent - LeetCode](#)

1. Brute Force

<https://leetcode.com/problems/orderly-queue/description/>

1. Brute Force

Follow Up:

2. Do it using min time complexity

[Make The String Great - LeetCode](#)

1. Brute Force

Follow Up:

2. Do it using constant Space

[Determine if Two Strings Are Close - LeetCode](#)

1. Brute Force

Follow Up:

2. Do it using constant Space

Graphs:

Implementation using adj List/Vector + Map

[BFS of graph | Practice | GeeksforGeeks](#)

[DFS of Graph | Practice | GeeksforGeeks](#)

[Flood Fill - LeetCode](#)

[Surrounded Regions - LeetCode](#)

[01 Matrix - LeetCode](#)

[Number of Enclaves - LeetCode](#)

[Number of Islands - LeetCode](#)

[Number of Distinct Islands | Practice | GeeksforGeeks](#)

[Find Eventual Safe States - LeetCode](#)

[Alien Dictionary | Practice | GeeksforGeeks](#)

[Shortest path in Directed Acyclic Graph | Practice | GeeksforGeeks](#)

[Negative weight cycle | Practice | GeeksforGeeks](#)

[Shortest Path in Undirected Graph | Practice | GeeksforGeeks](#)

[Minimum Genetic Mutation - LeetCode](#)

[Word Ladder - LeetCode](#)

[Map of Highest Peak - LeetCode](#)

[Rotting Oranges - LeetCode](#)

[Connected Components in an Undirected Graph | Practice | GeeksforGeeks](#)

[Network Delay Time - LeetCode](#)

[Shortest Path in Binary Matrix - LeetCode](#)

[Path With Minimum Effort - LeetCode****](#)

[Bellman-Ford | Practice | GeeksforGeeks](#)

[Floyd Warshall | Practice | GeeksforGeeks](#)

[Find the number of islands | Practice | GeeksforGeeks](#)

[Cheapest Flights Within K Stops - LeetCode](#)

[Is Graph Bipartite? - LeetCode](#)

[Problem - 687A - Codeforces](#)

[Snakes and Ladders - LeetCode](#)

[Detect Cycle using DSU | Practice | GeeksforGeeks](#)

[10 Interesting Problems of DSU\(Disjoint Set Union\) - Discuss - LeetCode](#)

[Satisfiability of Equality Equations - LeetCode](#)

[Redundant Connection - LeetCode](#)

[Number of Operations to Make Network Connected - LeetCode](#)

[Count Unreachable Pairs of Nodes in an Undirected Graph - LeetCode](#)

[Minimum Cost Path | Practice | GeeksforGeeks](#)

DP:

Optimization:

- Recursion
- Top Down
- Bottom Up

[Minimum steps to minimize n as per given condition | Practice | GeeksforGeeks](#)

(Do Implement top down and bottom up***)

[Coin Change - LeetCode](#)

[Maximum profit from sale of wines - GeeksforGeeks](#)

[Frog Jump | Practice | GeeksforGeeks](#)

Phase 1: 1D DP - Linear Problems

These are the foundations. If you struggle with these, everything else will be hard. Focus on understanding the transition from recursion to memoization to tabulation.

1. [Climbing Stairs](#)
2. [Min Cost Climbing Stairs](#)
3. [House Robber](#)
4. [House Robber II](#)
5. [Delete and Earn](#)
6. [Maximum Product Subarray](#)
7. [Jump Game](#)
8. [Jump Game II](#)
9. [Decode Ways](#)
10. [Decode Ways II](#)
11. [Fibonacci Number](#)
12. [N-th Tribonacci Number](#)
13. [Count Number of Ways to Place Houses](#)
14. [Solving Questions With Brainpower](#)
15. [Maximum Alternating Subsequence Sum](#)

Phase 2: 2D DP - Grid & Matrix

Grid DP is very common in interviews. Master the pattern of building solution bottom-up from `dp[i][j]`.

16. [Unique Paths](#)
17. [Unique Paths II](#)
18. [Minimum Path Sum](#)
19. [Triangle](#)
20. [Maximal Square](#)
21. [Dungeon Game](#)
22. [Cherry Pickup](#)
23. [Cherry Pickup II](#)
24. [Minimum Falling Path Sum](#)
25. [Minimum Falling Path Sum II](#)
26. [Maximal Rectangle](#)
27. [Count Square Submatrices with All Ones](#)

[Maximum path sum in matrix | Practice | GeeksforGeeks](#)

Phase 3: Knapsack Pattern

Knapsack is THE most important DP pattern. It appears in countless variations. Understand both 0/1 and unbounded knapsack.

28. [Partition Equal Subset Sum](#)
29. [Target Sum](#)
30. [Ones and Zeroes](#)
31. [Coin Change](#)
32. [Coin Change II](#)
33. [Perfect Squares](#)
34. [Minimum Cost For Tickets](#)
35. [Last Stone Weight II](#)
36. [Profitable Schemes](#)
37. [Tallest Billboard](#)

[Subset Sum Problem | Practice | GeeksforGeeks](#)

[Perfect Sum Problem | Practice | GeeksforGeeks](#)

[Partitions with Given Difference | Practice | GeeksforGeeks](#)

[Minimum sum partition | Practice | GeeksforGeeks](#)

[0 - 1 Knapsack Problem | Practice | GeeksforGeeks](#)

[Knapsack with Duplicate Items | Practice | GeeksforGeeks](#)

[Rod Cutting | Practice | GeeksforGeeks](#)

Do all Combination sum variations

Phase 4: String DP

String DP problems often involve LCS, LIS, or palindrome patterns. These are very common in interviews.

38. [Longest Common Subsequence](#)
39. [Longest Palindromic Substring](#)
40. [Longest Palindromic Subsequence](#)
41. [Palindromic Substrings](#)
42. [Edit Distance](#)
43. [Distinct Subsequences](#)
44. [Minimum ASCII Delete Sum for Two Strings](#)
45. [Delete Operation for Two Strings](#)
46. [Shortest Common Supersequence](#)
47. [Interleaving String](#)
48. [Regular Expression Matching](#)
49. [Wildcard Matching](#)

[Minimum Insertion Steps to Make a String Palindrome - LeetCode](#)
[Longest Common Substring | Practice | GeeksforGeeks](#)

Phase 5: Longest Increasing Subsequence

LIS is special because it has both $O(n^2)$ and $O(n \log n)$ solutions. Know both approaches.

50. [Longest Increasing Subsequence](#)
51. [Number of Longest Increasing Subsequence](#)
52. [Largest Divisible Subset](#)
53. [Russian Doll Envelopes](#)
54. [Maximum Length of Pair Chain](#)
55. [Longest String Chain](#)
56. [Minimum Number of Removals to Make Mountain Array](#)
57. [Delete Columns to Make Sorted III](#)

[Longest Bitonic subsequence | Practice | GeeksforGeeks](#)

Phase 6: Stock Problems

Stock problems are a special category that tests state machine DP. Learn the pattern once and all variations become easy.

- 58. [Best Time to Buy and Sell Stock](#)
- 59. [Best Time to Buy and Sell Stock II](#)
- 60. [Best Time to Buy and Sell Stock III](#)
- 61. [Best Time to Buy and Sell Stock IV](#)
- 62. [Best Time to Buy and Sell Stock with Cooldown](#)
- 63. [Best Time to Buy and Sell Stock with Transaction Fee](#)

Phase 7: Advanced Interview DP

These are harder problems that combine multiple patterns or require clever state definitions.

- 64. [Word Break](#)
- 65. [Word Break II](#)
- 66. [Palindrome Partitioning II](#)
- 67. [Burst Balloons](#)
- 68. [Stone Game IV](#)
- 69. [Maximum Score from Performing Multiplication Operations](#)
- 70. [Minimum Difficulty of a Job Schedule](#)

[Minimum Path Sum - LeetCode](#)

<https://leetcode.com/discuss/post/5190444/dynamic-programming-problems-category-wi-7xpk/>

<https://leetcode.com/discuss/post/1050391/must-do-dynamic-programming-problems-cat-g28m/>

TREES

[LeetCode - Binary Tree Maximum Path Sum](#)

[Path Sum - LeetCode****](#)

[Sum Root to Leaf Numbers - LeetCode](#)

[Diameter of Binary Tree - LeetCode](#)

[Binary Tree Preorder Traversal - LeetCode](#)

[Binary Tree Inorder Traversal - LeetCode****](#)

[Binary Tree Postorder Traversal - LeetCode](#)

[Binary Tree Level Order Traversal II - LeetCode](#)

[Maximum Depth of Binary Tree - LeetCode](#)

[Subtree of Another Tree - LeetCode](#)

[Same Tree - LeetCode](#)

[Balanced Binary Tree - LeetCode](#)

[Binary Tree Tilt - LeetCode](#)

[Binary Tree Zigzag Level Order Traversal - LeetCode](#)

[Tree Boundary Traversal | Practice | GeeksforGeeks](#)

[Vertical Order Traversal of a Binary Tree - LeetCode](#)

[Top View of Binary Tree | Practice | GeeksforGeeks](#)

[Bottom View of Binary Tree | Practice | GeeksforGeeks](#)

[Left View of Binary Tree | Practice | GeeksforGeeks](#)

[Binary Tree Right Side View - LeetCode](#)

[Symmetric Tree - LeetCode](#)

[Root to Leaf Paths | Practice | GeeksforGeeks](#)

[Lowest Common Ancestor of a Binary Tree - LeetCode*****](#)

[Path Sum II - LeetCode](#)

[Maximum Width of Binary Tree - LeetCode](#)

[Children Sum in a Binary Tree | Practice | GeeksforGeeks](#)

[Construct Binary Tree from Preorder and Inorder Traversal - LeetCode](#)

[Construct Binary Tree from Inorder and Postorder Traversal - LeetCode](#)

[Serialize and Deserialize Binary Tree - LeetCode](#)

[Mirror Tree | Practice | GeeksforGeeks](#)

BST

[Search in a Binary Search Tree - LeetCode](#)

[Floor in BST | Practice | GeeksforGeeks](#)

[Ceil in BST | Practice | GeeksforGeeks](#)

[Insert into a Binary Search Tree - LeetCode](#)

[Delete Node in a BST - LeetCode](#) *****

[Kth Smallest Element in a BST - LeetCode](#)

[Kth largest element in BST | Practice | GeeksforGeeks](#)

[Validate Binary Search Tree - LeetCode](#)

[Inorder Successor in BST | Practice | GeeksforGeeks](#)

[Binary Search Tree Iterator - LeetCode](#)

[Two Sum IV - Input is a BST - LeetCode](#)

[Construct Binary Search Tree from Preorder Traversal - LeetCode](#)

[Recover Binary Search Tree - LeetCode](#)

[Largest BST | Practice | GeeksforGeeks](#)

MORRIS TRAVERSAL FOR INORDER AND PREORDER

LINKED LIST:

[Reverse a Doubly Linked List | Practice | GeeksforGeeks](#)

[Insert in Middle of Linked List | Practice | GeeksforGeeks](#)

[Reverse Linked List - LeetCode](#)

[Add Two Numbers - LeetCode](#)

[Multiply two linked lists | Practice | GeeksforGeeks](#)

[Odd Even Linked List - LeetCode](#)

[Sort a linked list of 0s, 1s and 2s | Practice | GeeksforGeeks](#)

[Remove Nth Node From End of List - LeetCode](#)

[Delete the Middle Node of a Linked List - LeetCode](#)

[Palindrome Linked List - LeetCode](#)

[Add 1 to a Linked List Number | Practice | GeeksforGeeks](#)

[Intersection of Two Linked Lists - LeetCode](#)

[Linked List Cycle II - LeetCode](#)

[Find length of Loop | Practice | GeeksforGeeks](#)

[Maximum Twin Sum of a Linked List - LeetCode](#)

[Remove Duplicates from Sorted List II - LeetCode](#)

[Reverse Nodes in k-Group - LeetCode](#)

[Rotate List - LeetCode](#)

[Merge Two Sorted Lists - LeetCode](#)

[Merge k Sorted Lists - LeetCode](#)

[Reorder List - LeetCode](#)

[Sort a k sorted doubly linked list | Practice | GeeksforGeeks](#)

STACK && QUEUE

Master NSE,PSE,NGE,PGE

Infix to postfix and all variations

Implementation of all variations

[Sum of Subarray Minimums - LeetCode](#)

[Sum of Subarray Ranges - LeetCode](#)

[Asteroid Collision - LeetCode](#) ***

[Largest Rectangle in Histogram - LeetCode](#)

[Remove K Digits - LeetCode](#)

[Maximal Rectangle - LeetCode](#)

[Online Stock Span - LeetCode](#)

[The Celebrity Problem | Practice | GeeksforGeeks](#)

[LRU Cache - LeetCode](#)***

[Minimum Remove to Make Valid Parentheses - LeetCode](#)

Sliding Window (V.V.IMP)

FIXED SIZE:

[First negative in every window of size k | Practice | GeeksforGeeks](#)

[K Sized Subarray Maximum | Practice | GeeksforGeeks](#)

[Max Sum Subarray of size K | Practice | GeeksforGeeks](#)

DYNAMIC SIZE:

[Smallest window containing 0, 1 and 2 | Practice | GeeksforGeeks](#)

[Minimum Size Subarray Sum - LeetCode](#)

[Longest Substring with K Uniques | Practice | GeeksforGeeks](#)

[Length of Longest Subarray With at Most K Frequency - LeetCode](#)

[Subarray Product Less Than K - LeetCode](#)

[Find All Anagrams in a String - LeetCode](#)

SEGMENT TREES:

https://www.youtube.com/watch?v=sSwR9h-c4Wo&list=PL-Jc9J83PlIFy1445K_x1M_O3j9kr3FI

CF STUFF

<https://themecp.vercel.app/contest>